

RU Digital — da especificação ao padrão no código

Roteiro conceitual · ATV RU Digital v2 · Arquitetura, princípios e padrões de projeto

Como usar este roteiro. Não haverá aula expositiva sobre esta atividade: **este texto é a aula**. Leia na ordem, com calma (umas 2 horas no total), antes de abrir o roteiro prático. Os cinco blocos seguem uma única cadeia — requisito → decisão → estrutura → código → padrão — e o **roteiro prático** remete a eles pelo nome (“leia o Bloco E.3”). Os exemplos resolvidos estão aqui e no esqueleto de código; quando algo parecer ambíguo, **os testes decidem**.

A Da frase à especificação

B Da especificação à decisão

C Da decisão à estrutura

D Da estrutura ao código

E Do princípio ao padrão

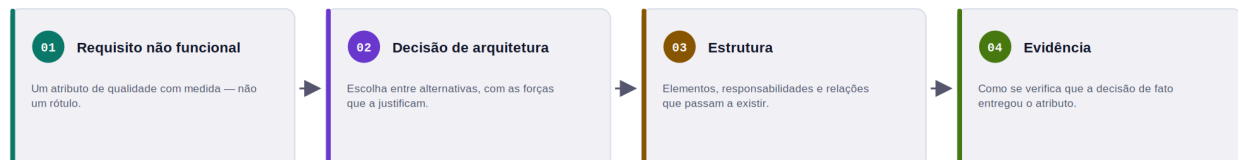
+ Anexo · especificação v2

O fio do roteiro

Uma decisão de arquitetura não nasce do gosto de quem projeta. Ela nasce de um **requisito não funcional com medida**, escolhe entre **alternativas**, cria **estrutura** — e só é real se houver **evidência** de que o sistema a cumpre. Os Blocos A a D seguem essa cadeia com um exemplo resolvido: a catraca que funciona sem internet. O Bloco E mostra como os **padrões de projeto** da aula 06 dão forma, dentro do código, às decisões que vocês vão tomar nos quatro pedidos da atividade.

DA EXIGÊNCIA À ESTRUTURA

O que uma decisão de arquitetura encadeia



O MESMO EXEMPLO ATRAVESSANDO A CADEIA · RU DIGITAL, REQUISITO R3

- 01** "Com o enlace indisponível, a catraca deve autorizar ou negar a entrada em até 1 s a partir de réplica local, e sincronizar as passagens em até 5 min após o restabelecimento."
- 02** Processar a autorização na borda: a catraca deixa de ser terminal burro e passa a ser um nó com decisão própria. Descarta cliente-servidor síncrono.
- 03** Réplica local de saldos na catraca · fila persistente de passagens · serviço de sincronização · política de resolução de conflito.
- 04** Teste de injeção de falha: derruba-se o enlace, mede-se o tempo de decisão e verifica-se se a passagem aparece no servidor dentro da janela.

A cadeia que organiza o roteiro: requisito → decisão → estrutura → evidência.

Requisito vago não vira arquitetura — vira discussão. Requisito com medida vira decisão; decisão vira fronteira; o padrão dá forma à fronteira; e ela só existe se o código a respeitar.

Roteiro de leitura

BLOCO	TEMPO	O QUE VOCÊ DEVE SABER FAZER AO FINAL
A · Da frase à especificação	20 min	reconhecer requisito vago e ler um cenário de qualidade com medida
B · Da especificação à decisão	20 min	comparar alternativas contra cenários e escrever um ADR
C · Da decisão à estrutura	15 min	dizer em que nível do C4 a decisão aparece e desenhar o nível 3 em Mermaid
D · Da estrutura ao código	25 min	identificar o segredo de uma classe e escrever uma regra de fronteira
E · Do princípio ao padrão	40 min	escolher, pelo problema, entre Adaptador, Fábrica, Strategy, Decorador e Observador

Da frase à especificação

Revisão de requisitos (ES1), aplicada ao caso. Aqui não há tarefa: o objetivo é chegar à **especificação v2**, que é a base de tudo o que vem depois.

A.1 · O sistema e como os requisitos chegaram

O restaurante universitário (RU) vende créditos pelo aplicativo e cobra por um gateway de pagamento externo. Na porta, uma catraca lê a carteirinha ou um QR code e decide se a pessoa entra, conforme o saldo e o vínculo (aluno, servidor, visitante, bolsista integral). A nutricionista publica o cardápio da semana. Os requisitos chegaram assim:

R1	O sistema deve permitir que o usuário compre créditos pelo aplicativo.
R2	O sistema deve ser rápido.
R3	A catraca deve funcionar mesmo quando a internet cair.
R4	O sistema deve ser seguro.
R5	O sistema deve exibir o cardápio da semana.
R6	O sistema deve ser fácil de manter.

R1 e R5 são **funcionais**: dizem o que o sistema faz. R2, R3, R4 e R6 são **não funcionais**: restringem *como* ele pode ser construído. São os não funcionais que chegam até a arquitetura.

A.2 · Três defeitos, e só três

DEFEITO	O QUE É	NO RU
Não verificável	Não há teste que passe ou falhe	R2 “rápido”, R6 “fácil de manter”
Ambíguo	Admite leituras incompatíveis	R4 “seguro” — protege o crédito, ou protege a pessoa?
Incompleto	Verificável na intenção, mas faltam limites para projetar	R3 — sem rede por quanto tempo? e quando a rede volta?

A.3 · Por que isso trava a arquitetura

Um requisito vago **não restringe nada**: qualquer arquitetura “atende” a “o sistema deve ser rápido”. Sem restrição não há critério para escolher entre alternativas — e a escolha vira preferência. É por isso que a especificação vem antes da decisão.

A.4 · Ferramenta 1 — o cenário de qualidade em seis partes

Formato de Bass, Clements e Kazman. Transforma um adjetivo em algo que um teste consegue verificar.

R3 REESCRITO COMO CENÁRIO	
Fonte	Infraestrutura de rede do câmpus
Estímulo	O enlace entre o prédio do RU e o servidor central fica indisponível
Artefato	App da Catraca
Ambiente	Operação normal, horário de pico, fila formada
Resposta	Autoriza ou nega a entrada a partir de dados locais; registra cada passagem; envia tudo ao servidor quando o enlace volta
Medida	Decisão em ≤ 1 s · passagens no servidor em ≤ 5 min após o restabelecimento · nenhuma passagem perdida

A parte que mais falha na prática é a **medida**. “Rapidamente” não é medida; “ ≤ 1 s” é. Em requisitos de manutenibilidade, a medida costuma ser *quantos arquivos* uma mudança toca, ou *quais classes não podem mudar* — é assim que o RNF5, o RNF9 e o RNF10 estão escritos.

A.5 · Ferramenta 2 — o modelo de qualidade da ISO/IEC 25010:2023

Dá nome ao atributo e evita que cada pessoa chame a mesma coisa de um jeito. A versão de 2023 tem nove características:

CARACTERÍSTICA	SUBCARACTERÍSTICAS (SELEÇÃO)	NO RU
Adequação funcional	completude, correção, apropriação	R1, R5
Eficiência de desempenho	comportamento temporal, capacidade, uso de recursos	R2
Compatibilidade	coexistência, interoperabilidade	integração com o sistema acadêmico
Capacidade de interação 2023	aprendizado, operabilidade, inclusividade , proteção contra erro	quem não tem celular?
Confiabilidade	ausência de falhas, disponibilidade, tolerância a falhas , recuperabilidade	R3
Segurança (<i>security</i>)	confidencialidade, integridade, não repúdio, responsabilização, autenticidade , resistência	R4 (crédito), bolsista
Manutenibilidade	modularidade, reusabilidade, analisabilidade, modificabilidade , testabilidade	R6 (RNF5, RNF9, RNF10)
Flexibilidade 2023	adaptabilidade, escalabilidade, instalabilidade, substituíbilidade	—
Segurança física (<i>safety</i>) 2023	restrição operacional, identificação de risco, falha segura , alerta de perigo	R4 (pessoa)

Boa parte do material em português na internet ainda descreve a versão de 2011, com oito características: “usabilidade” virou *capacidade de interação*, “portabilidade” foi absorvida por *flexibilidade*, e *safety* ganhou característica própria. Há fontes secundárias que divergem sobre onde ficou a testabilidade; este roteiro a mantém em manutenibilidade.

A.6 · Ferramenta 3 — perguntar ao stakeholder

Especificar gera perguntas; perguntar melhora a especificação. Estas foram as perguntas que a reescrita levantou, e as respostas da gestão do RU:

PERGUNTA	RESPOSTA	VIROU
O que é “rápido”? Em que momento?	No pico do almoço chegam 800 pessoas usando o app ao mesmo tempo. Comprar não pode passar de 2 s.	RNF1
Quanto tempo sem rede? E quando volta?	Já ficamos uma manhã inteira sem enlace. Quando voltar, em 5 min o servidor precisa saber de todo mundo que passou.	RNF2
“Seguro” contra o quê?	As duas coisas: ninguém pode mexer no saldo sem deixar rastro; e em emergência a catraca não pode prender ninguém.	RNF3 e RNF4
“Fácil de manter” contra qual mudança?	O contrato com o gateway de pagamento vence no ano que vem, e o próximo fornecedor já foi escolhido. E o conselho muda a regra de entrada de tempos em tempos.	RNF5 e RNF9
Quem precisa saber quando o cardápio sai?	O app, o site e a gestão por e-mail. O DCE já pediu para ser avisado também, e outros vão pedir.	RNF10
Quanto paga quem não é aluno?	Servidor e visitante pagam mais. Bolsista integral não paga nada.	RF4
E como se sabe que quem não paga é o bolsista?	Hoje não se sabe. Tem carteirinha de bolsista sendo emprestada — é o nosso maior problema.	RNF8
Quem vê o consumo de cada pessoa?	Ninguém individualmente. A gestão só quer números agregados.	RNF6

Repare nas linhas do RNF8 e do RNF6: **são requisitos que não existiam em nenhum R original**. Nenhuma reescrita de frase os teria encontrado — só a pergunta.

A.7 · O resultado: a especificação v2 e os quatro pedidos

A especificação completa está no [Anexo](#). É ela que a atividade usa. O RNF2 é o exemplo resolvido deste roteiro; os outros quatro são os **pedidos de mudança** que vocês vão atender, na ordem:

PEDIDO	REQUISITO	EM UMA FRASE
exemplo	RNF2	a catraca decide sem rede — resolvido nos Blocos B a D
1	RNF5	trocar o gateway de pagamento mexendo em um só arquivo
2	RNF9	nova regra de entrada (tolerar saldo negativo até R\$ 5,00) sem mexer na Catraca
3	RNF8	a isenção do bolsista só vale para o titular, uma vez por refeição
4	RNF10	avisar novos interessados quando o cardápio sai, sem mexer no Cardapio

Da especificação à decisão

Como se escolhe uma arquitetura sem ser por gosto: alternativas avaliadas contra os mesmos cenários, e um registro que diz o que se ganhou e o que se pagou.

B.1 · O que é uma decisão arquitetural

É uma decisão cujo custo de mudar **cresce com o tempo** e **atravessa vários módulos**. Trocar a biblioteca de gráficos não é; decidir onde a catraca processa a autorização é. Entre as duas há as decisões de **projeto** — como organizar as classes dentro de um componente — que são as que os padrões do Bloco E resolvem. Nesta atividade vocês registram as duas do mesmo jeito, porque o método é o mesmo.

B.2 · Duas candidatas, os mesmos cenários

Para o RNF2 há duas alternativas naturais. **A · cliente-servidor síncrona**: a catraca pergunta ao servidor a cada passagem. **B · autorização na borda**: a catraca decide sozinha, com uma cópia local dos dados. Avaliá-las “no abstrato” não leva a nada; avalia-se contra os cenários da especificação — e contra os custos de construir e operar.

AVALIAÇÃO			
Duas candidatas, os mesmos cenários			
Arquitetura não se avalia no abstrato: avalia-se contra cenários de qualidade. Onde as colunas discordam, há um trade-off.			
CENÁRIO DE QUALIDADE	A · Cliente-servidor síncrona	B · Autorização na borda	TRADE-OFF
R2 · compra em até 2 s no pico de 800 usuários	atende	atende	
R3 · entrada decidida em até 1 s com o enlace caído	não atende	atende	●
Custo de construção do primeiro release	menor	maior	●
Esforço para depurar um saldo divergente	baixo	alto	●
Tempo de indisponibilidade percebido na porta	alto	quase nulo	●
Complexidade de implantação e monitoramento	baixa	alta	●
PONTO DE SENSIBILIDADE Uma decisão que afeta fortemente UM atributo. Aqui: onde a autorização é processada.		PONTO DE TRADE-OFF A mesma decisão melhora um atributo e piora outro. Aqui: disponibilidade sobe, manutenibilidade e simplicidade caem.	

Onde as colunas discordam, há um trade-off.

B.3 · Ponto de sensibilidade e ponto de trade-off

Ponto de sensibilidade

Uma decisão que afeta fortemente *um* atributo. Aqui: *onde a autorização é processada* determina a disponibilidade na porta.

Ponto de trade-off

A mesma decisão melhora um atributo e piora outro. Aqui: disponibilidade sobe; manutenibilidade e simplicidade caem.

Os dois termos vêm do método ATAM, do SEI. Não existe decisão sem ponto de trade-off; existe decisão que não o declarou.

B.4 · O registro de decisão (ADR)

Proposto por Michael Nygard em 2011: um arquivo curto por decisão, versionado junto com o código. Quatro partes:

PARTE	PERGUNTA QUE RESPONDE
Contexto	O que é verdade no problema e obriga a decidir? Cite o cenário e a medida.
Decisão	O que foi escolhido — uma <i>estrutura</i> .
Consequências	O que se ganha, o que se paga , que estrutura nasce, e qual evidência prova que o código cumpre.
Alternativa descartada	Qual, e por quê.

ADR-001 · Autorização de entrada na borda (completo em `docs/decisoes/` no esqueleto)

Contexto. O enlace do prédio do RU é instável e o RNF2 exige decisão em ≤ 1 s sem ele, e sincronização em ≤ 5 min quando voltar.

Decisão. Processar a autorização na catraca, a partir de uma réplica local de saldos, registrando cada passagem numa fila persistente que um sincronizador envia ao servidor.

Consequências. **Ganha** disponibilidade na porta e latência baixa. **Paga** estado duplicado, um caminho de sincronização a mais, saldo temporariamente divergente; duas catracas sem enlace não se enxergam. Nasce: Réplica, Fila e Sincronizador. Evidência: só o Sincronizador pode usar o pacote `ru.rede` (teste de fronteira).

Descartada. Cliente-servidor síncrona: mais simples, mas a catraca para quando o enlace cai.

B.5 · Três erros que invalidam um ADR

- **Decisão que é só um nome.** “Usar SQLite” ou “usar o padrão Strategy” não é decisão; é rótulo. A decisão é a estrutura (“a Catraca recebe a regra de entrada pronta, e só a montagem sabe qual é”). O nome do padrão pode — e deve — aparecer, mas *depois* da estrutura, como vocabulário.
- **Consequência sem custo.** Se nada piorou, a análise não foi feita. Toda decisão tem uma linha *paga*. Padrões também custam: mais classes, mais indireção.
- **Contexto sem cenário.** Se o contexto não cita a medida da especificação, a decisão não tem como ser avaliada.

Da decisão à estrutura

Onde a decisão aparece no desenho. O modelo C4, de Simon Brown, organiza a arquitetura em níveis de zoom — cada um para um público.

C.1 · Os quatro níveis do C4

DOCUMENTAÇÃO

O modelo C4: quatro níveis de zoom

Mesma arquitetura, quatro escalas. Você escolhe o nível pelo público — e quase nunca precisa dos quatro.



Mesma arquitetura, quatro escalas. No curso: níveis 1 e 2 sempre; nível 3 no contêiner que a decisão tocou; nível 4 nunca.

C.2 · Nível 1 · contexto

RU DIGITAL · C4 NÍVEL 1

Diagrama de contexto

O sistema como caixa preta. Serve para acordar fronteira e responsabilidade — inclusive com quem não é técnico.

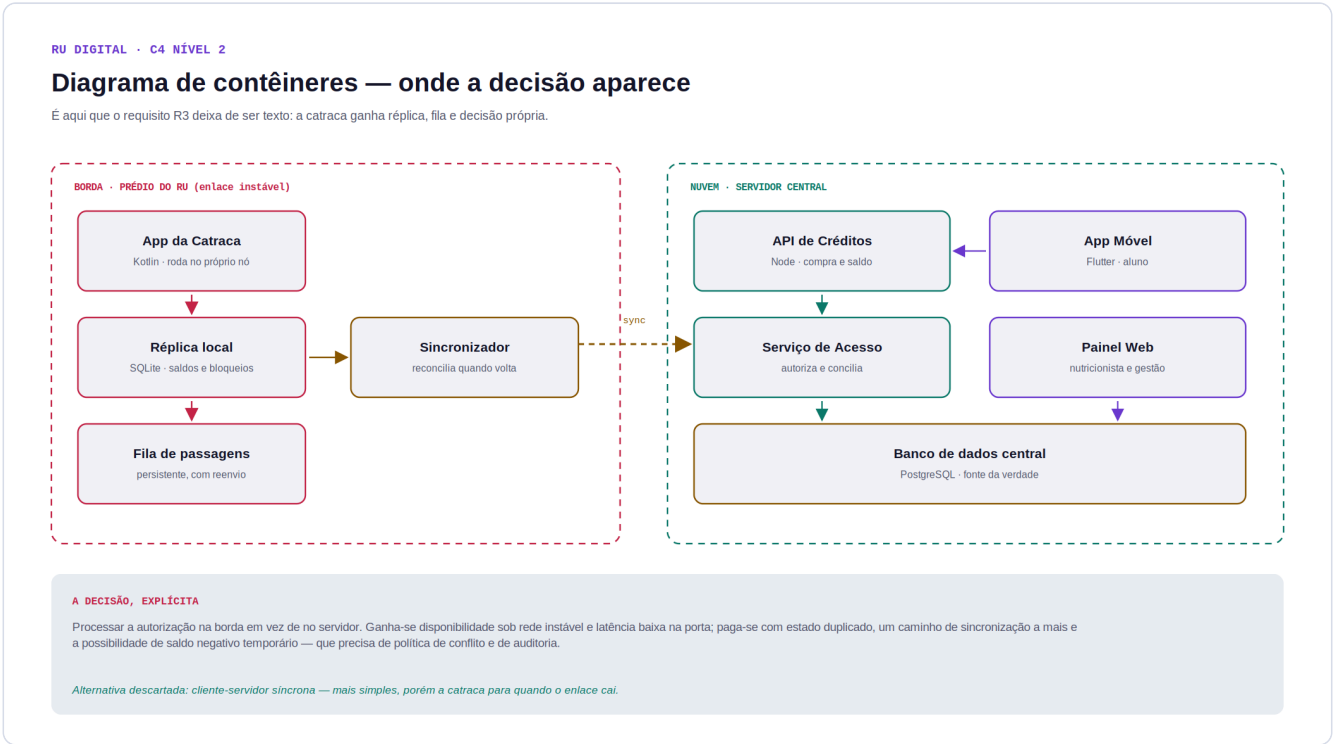


A PERGUNTA QUE ESTE NÍVEL RESOLVE

O que é responsabilidade nossa e o que é de terceiro? Note que a catraca aparece como sistema externo — e essa classificação é, ela própria, uma decisão que o nível 2 vai colocar em xeque.

No contexto, a catraca aparece como sistema externo.

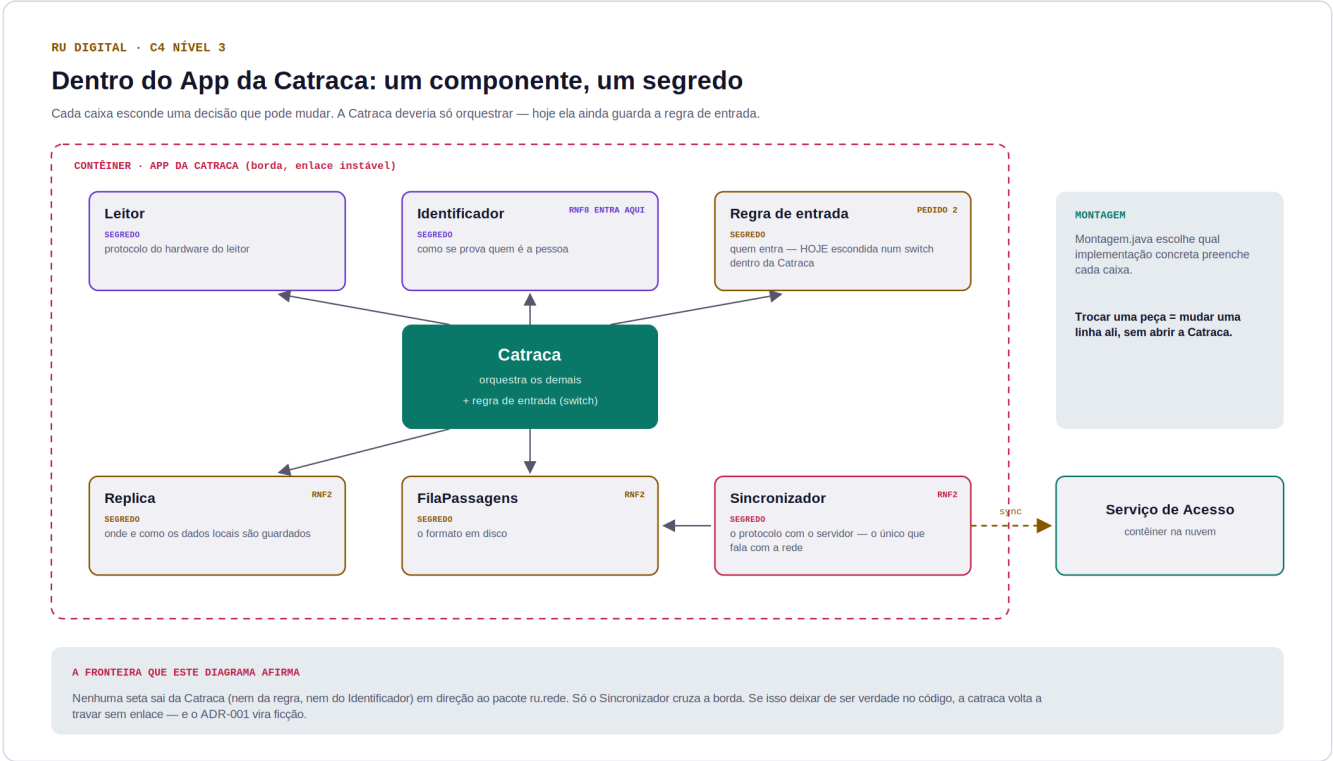
C.3 · Nível 2 · contêineres — onde a decisão do ADR-001 aparece



A decisão trouxe a catraca para dentro do sistema: réplica, fila e sincronizador são contêineres nossos.

Compare os dois níveis: **um requisito não funcional mudou a fronteira do sistema**. A catraca era de terceiro; passou a ser nossa, porque precisa decidir sozinha.

C.4 · Nível 3 · componentes do App da Catraca



Cada componente guarda um segredo. A Catraca deveria só orquestrar — o pedido 2 corrige isso.

Nem toda decisão cria contêiner. Algumas mudam a fronteira do sistema (nível 2); outras reorganizam o interior de um contêiner que já existe (nível 3). As decisões dos quatro pedidos são todas do segundo tipo: nenhum processo novo, nenhum banco novo — só interfaces e classes novas dentro de um contêiner. Ao registrar uma decisão, diga em qual nível ela aparece e por quê.

C.5 · Como desenhar: Mermaid

Os diagramas da atividade são escritos em texto, versionados com o código. O mínimo necessário:

```

%% mermaid
flowchart LR
    cat["Catraca"]
    regra["«interface» RegraEntrada<br/><small>[RNF9]</small>"]
    s["Serviço externo"]
    cat --> regra
    x -. implementa .-> regra

```

A seta vai **de quem usa para quem é usado**. Entre colchetes, o cenário que justifica a caixa. Uma interface se marca com `<interface>` no rótulo. Para visualizar: mermaid.live, o VS Code com a extensão *Markdown Preview Mermaid Support*, ou o GitHub. Exemplo completo em `docs/c4-catraca.md`.

Da estrutura ao código

A fronteira existe duas vezes: no diagrama e no código. Os princípios de projeto são o que mantém a segunda igual à primeira.

D.1 · Classe não é um pedaço do fluxo — é um segredo

Parnas (1972): o critério para decompor um sistema é **esconder as decisões que podem mudar**. Cada classe da catraca guarda uma:

CLASSE	SEGREDO	MUDANÇA QUE ELA ABSORVE
Leitor	protocolo do hardware	fabricante novo
Replica	como os dados locais são guardados	mapa em memória → SQLite
FilaPassagens	o formato em disco	memória → arquivo
Sincronizador	o protocolo com o servidor	HTTPS → MQTT
IdentificadorPorCarteirinha	como se prova quem é a pessoa	(é aqui que o pedido 3 mexe)
Montagem	quais implementações concretas são usadas	trocar uma peça por outra
Catraca	deveria ser nenhum — só orquestrar	hoje ainda guarda a regra de entrada (pedido 2)

Um teste simples: “recebe a leitura” é etapa, não segredo. “O formato em que a leitura chega” é segredo.

D.2 · Quem escolhe as peças: injeção de dependência

A `Catraca` não cria seus colaboradores; recebe-os prontos no construtor. Quem escolhe é um único lugar, `src/ru/catraca/Montagem.java`:

```
public static Catraca criarCatraca(Replica replica, FilaPassagens fila,
                                   Relogio relógio, String regra) {
    return new Catraca(new Leitor(), new IdentificadorPorCarteirinha(),
                       replica, fila, relógio, regra);
}
```

A `Catraca` conhece só o **contrato** de cada peça — por exemplo, a interface `Identificador`, cujo método `identificar(credencial)` devolve uma matrícula ou `null`. Qualquer objeto que cumpra esse contrato serve. Os testes de aceitação criam os objetos **sempre pelos métodos de montagem** (`Montagem.criarCatraca`, `MontagemCredito.criarServicoCompra`, `MontagemCardapio.criar`): é por isso que vocês podem criar as classes que quiserem, com os nomes que quiserem, desde que a montagem as encaixe.

D.3 · Os princípios, no caso

PRINCÍPIO	O QUE DIZ	ONDE APARECE NA CATRACA
Ocultamento de informação	cada módulo esconde uma decisão que pode mudar	a tabela de D.1
Acoplamento aceitável	serviço útil + interface estável + acesso só pela interface	a <code>Catraca</code> usa a réplica só por <code>usuario()</code> , <code>tarifa()</code> , <code>debitar()</code>
Inversão de dependências	dependa de abstrações, não de implementações	a <code>Catraca</code> depende da interface <code>Identificador</code> , não de <code>IdentificadorPorCarteirinha</code>
Aberto/fechado	estender sem modificar	uma peça nova entra pela <code>Montagem</code> , sem editar a <code>Catraca</code>
Composição sobre herança	“possui” acopla menos que “é um”	um objeto pode <i>envolver</i> outro com o mesmo contrato e acrescentar comportamento

Vazamento pela porta da frente. Uma classe pode esconder seus atributos (`private`) e ainda vaziar o segredo pelo que devolve ou pelo que usa. Se o nome, o tipo ou um código de status de um fornecedor (`Status.PAID` , `chargeId`) aparece no meio da regra de negócio, o segredo vazou — trocar o fornecedor obriga a reescrever a regra.

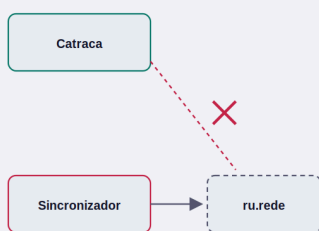
D.4 · A fronteira existe duas vezes

PRINCÍPIOS DE PROJETO · EVIDÊNCIA

A fronteira existe duas vezes: no diagrama e no código

Uma regra de fronteira transforma a seta que NÃO existe no diagrama em um teste que falha quando alguém a cria.

NO DIAGRAMA (ADR-001)



A seta que não pode existir. No desenho ela é só ausência — ninguém é avisado se aparecer.

NO CÓDIGO · test/ru/testes/TestesFronteira.java

```
public void testCatracaSoSincronizadorFalaComARede() {
    Fronteiras.naoPodeUsar("ru.catraca", // quem é vigiado
        new String[]{"ru.rede"}, // o que não pode usar
        "ru.catraca.Sincronizador"); // quem está liberado
}
```

\$./rodar.sh testes fronteira - 2 ok, 0 falhas

E SE ALGUÉM CRIAR O ATALHO?

```
// src/ru/catraca/Catraca.java
import ru.rede.ClienteHttp; // “só para conferir o saldo online”

FALHA TestesFronteira.testCatracaSoSincronizadorFalaComARede
Fronteira violada:
ru.catraca.Catraca -> ru.rede.ClienteHttp
```

POR QUE ISSO É PRINCÍPIO DE PROJETO

Ocultamento de informação diz que cada módulo guarda um segredo; acoplamento aceitável diz que se acessa o outro só pela interface. A regra de fronteira é o que impede o atalho que viola os dois — é o antidoto do Big Ball of Mud, um atalho de cada vez. Na literatura, testes assim são chamados de funções de aptidão da arquitetura (Ford, Parsons e Kua).

A regra de fronteira é a evidência, no código, da decisão registrada no ADR.

No diagrama, uma fronteira é uma seta que *não existe* — e ninguém é avisado se ela aparecer. No código, uma **regra de fronteira** é um teste que lê as dependências das classes (`import` , nomes qualificados e classes do mesmo pacote) e falha quando alguém cria o atalho. Na literatura, testes que protegem uma característica da arquitetura se chamam **funções de aptidão** (Ford, Parsons e Kua).

D.5 · As ferramentas do esqueleto

O esqueleto é Java puro (JDK 17 ou mais novo), sem Maven nem JUnit. Na pasta do projeto, use `rodar` no Windows e `./rodar.sh` no Linux/macOS. No **PowerShell** (prompt começando com `PS`), escreva `.\rodar` — o PowerShell não executa scripts da pasta atual sem o `.\` na frente:

COMANDO	PARA QUÊ
<code>rodar testes</code>	compila e roda todos os testes; verde = cumprido
<code>rodar testes pedido2</code>	só uma suíte: <code>funcionais</code> , <code>fronteira</code> , <code>pedido1</code> ... <code>pedido4</code>
<code>rodar simular --sem-rede</code>	um almoço com o enlace caído
<code>rodar grafo ru.creditos</code>	as dependências <i>reais</i> de um pacote, lidas do código
<code>rodar diff --marcar</code> e <code>rodar diff --desde-marco</code>	quais arquivos mudaram desde um marco

Uma regra de fronteira se escreve em `test/ru/testes/TestesFronteira.java`, com `Fronteiras.naoPodeUsar`. Todo método público cujo nome começa com `test` vira um teste:

```
public void testCatracaSoSincronizadorFalaComARede() {  
    Fronteiras.naoPodeUsar("ru.catraca",           // quem é vigiado (pacote ou classe)  
        new String[]{"ru.rede"},                  // o que não pode ser usado  
        "ru.catraca.Sincronizador");              // exceções (zero ou mais)  
}
```

Os nomes são prefixos: `"ru.catraca"` vale para o pacote e seus subpacotes; `"ru.catraca.Sincronizador"` vale para uma classe só.

Princípio sem o problema que ele resolve é cerimônia. Cada pedido da atividade traz um problema concreto: descubram qual princípio ele pede — o Bloco E mostra que forma esse princípio costuma tomar.

Do princípio ao padrão

Um padrão de projeto é um princípio aplicado a um problema recorrente, com nome. Este bloco revisa os padrões da aula 06 que resolvem os quatro pedidos — cada um com o problema, a estrutura e o código antes e depois.

E.1 · O que é (e o que não é) um padrão

Gamma, Helm, Johnson e Vlissides (1994), o “livro da Gangue dos Quatro” (GoF), descrevem cada padrão por quatro elementos: **nome**, **problema** (quando aplicar), **solução** (os papéis e como colaboram) e **consequências** (o que se ganha e o que se paga). Um padrão **não** é biblioteca nem código pronto: é uma forma que se adapta ao caso. Os 23 padrões do GoF se dividem em três famílias:

FAMÍLIA	PERGUNTA QUE RESPONDE	NESTA ATIVIDADE
Criação	quem decide qual classe concreta é instanciada?	Fábrica (pedido 1)
Estrutural	como encaixar peças que não foram feitas para se encaixar?	Adaptador (pedido 1), Decorador (pedido 3)
Comportamental	quem faz o quê, e quem avisa quem?	Strategy (pedido 2), Observador (pedido 4)

A ordem importa. Parte-se do problema (o pedido e o seu cenário com medida) para o princípio violado, e só então para o padrão. O caminho contrário — “vamos usar Strategy aqui” — produz cerimônia: classes que ninguém precisava. Um padrão só se justifica pela **mudança que ele absorve**.

E.2 · O mapa: cada pedido, um padrão

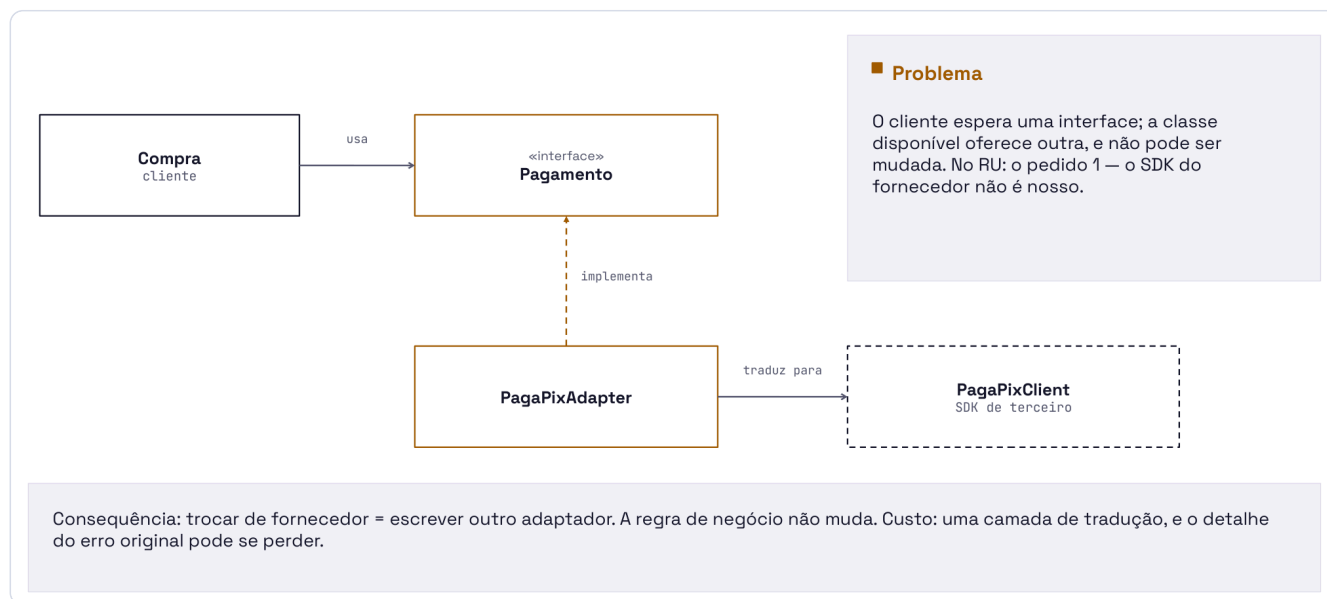
PEDIDO	PADRÃO	COMO RESPONDE	PRINCÍPIO
1 · trocar o gateway (RNF5)	Adaptador + Fábrica	a compra fala com um contrato nosso (<code>Pagamento</code>); cada fornecedor ganha um adaptador; a fábrica escolhe qual	ocultamento + inversão de dependências
2 · nova regra de entrada (RNF9)	Strategy	a <code>Catraca</code> usa uma <code>RegraEntrada</code> trocável	aberto/fechado
3 · bolsista titular, uma vez por refeição (RNF8)	Decorador	uma regra envolve a regra existente e acrescenta a restrição	aberto/fechado + composição
4 · avisar sobre o cardápio (RNF10)	Observador	o cardápio avisa quem assinou, sem conhecê-los	acoplamento fraco

Nos quatro casos, a mudança entra por **código novo** — não por edição do que já funcionava. Os nomes de classes nesta tabela e nos exemplos abaixo são sugestões: os testes não exigem nenhum deles.

E.3 · Pedido 1 · Adaptador e Fábrica

Problema. A classe `ServicoCompra` fala a língua da PagaPix: chama `createCharge` em centavos, compara com `Status.PAID`, guarda o `chargeId`. O novo fornecedor, CobraFácil, tem outra API: recebe `BigDecimal`, devolve `RetornoCobranca` e lança uma exceção verificada em falha técnica. O SDK de terceiro **não pode ser mudado**; a regra de crédito **não deve** mudar.

Adaptador (GoF, estrutural). Intenção: converter a interface de uma classe na interface que o cliente espera. O cliente passa a depender de um contrato *nosso*; cada fornecedor ganha uma classe que traduz esse contrato para o SDK dele — valores, status e exceções.



Adaptador: a regra de crédito depende do contrato Pagamento; o vocabulário do fornecedor fica preso no adaptador.

Antes — o vocabulário do fornecedor na regra

```
class ServicoCompra {
    PagaPixClient pagapix = new PagaPixClient(..);

    Recibo comprar(String mat, BigDecimal v, ..) {
        PagaPixResponse r = pagapix.createCharge(
            v.movePointRight(2).longValueExact(), ..);
        if (r.status() == Status.PAID) { ... }
    }
}
```

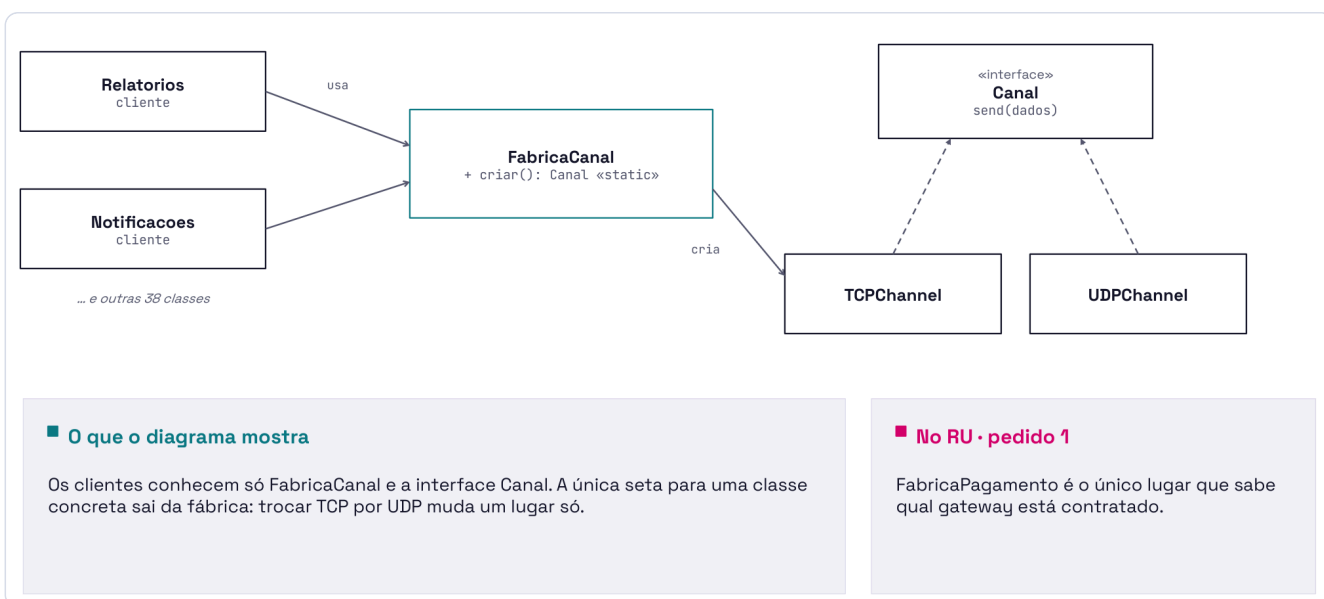
Depois — o contrato é nosso

```
interface Pagamento {
    ResultadoPagamento cobrar(String mat,
        BigDecimal v, String token);
}

class PagaPixAdapter implements Pagamento {
    // aqui (e só aqui) moram centavos, PAID...
}

class ServicoCompra {
    ServicoCompra(.., Pagamento pagamento) { .. }
}
```

Fábrica. Alguém ainda precisa dizer `new PagaPixAdapter(..)` ou `new CobraFacilAdapter(..)`. Se esse `new` se espalhar, trocar de fornecedor volta a exigir edições em vários lugares. A **fábrica simples** concentra a criação num lugar só: recebe o nome do gateway e devolve um `Pagamento`.



Fábrica simples: os clientes conhecem só a fábrica e a interface; a única seta para uma classe concreta sai da fábrica.

```

final class FabricaPagamento {
    static Pagamento criar(String gateway) {
        return switch (gateway) {
            case "pagapix" → new PagaPixAdapter(...);
            case "cobrafacil" → new CobraFacilAdapter(...);
            default → throw new IllegalArgumentException("gateway desconhecido: " + gateway);
        };
    }
}
  
```

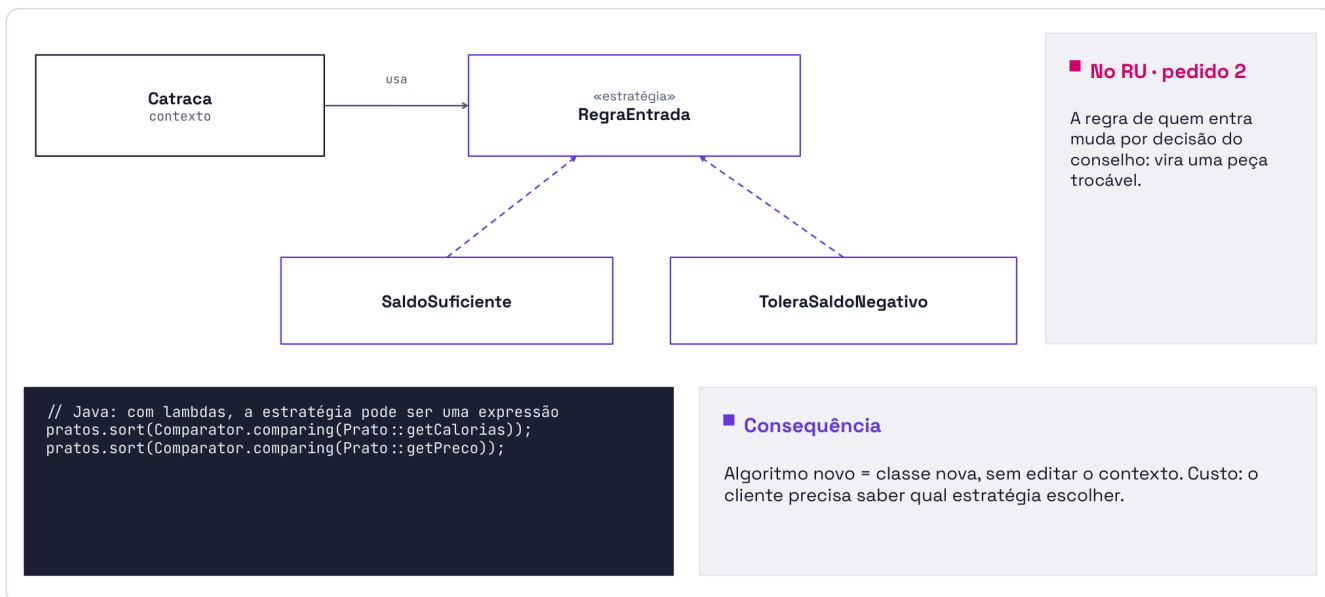
Um **switch** na fábrica é aceitável; na regra de negócio, não. A fábrica existe justamente para ser o único lugar que conhece as classes concretas. As variações do GoF — **Factory Method** (a subclasse decide o que criar) e **Abstract Factory** (uma família de objetos que combinam) — resolvem problemas maiores que este; aqui, a fábrica simples basta. Use a mais simples que resolva.

Consequências. **Ganha:** fornecedor novo = um adaptador + uma linha na fábrica; a regra de crédito fica testável sem SDK. **Paga:** mais classes; uma tradução a manter por fornecedor; o contrato só oferece o que é comum a todos (o detalhe do erro original pode se perder).

E.4 · Pedido 2 · Strategy

Problema. A **Catraca** recebe um texto da configuração e decide quem entra com um **switch**. O conselho aprovou uma regra nova (**TOLERA_NEGATIVO** : entra quem fica com saldo de até -R\$ 5,00) — e no ano que vem pode aprovar outra. Cada regra nova reabre a classe mais crítica da porta.

Strategy (GoF, comportamental). Intenção: definir uma família de algoritmos, encapsular cada um e torná-los intercambiáveis. O “contexto” (a **Catraca**) recebe a estratégia pronta e só a chama.



Strategy: a Catraca depende de RegraEntrada; cada regra é uma classe; a escolha é da montagem.

Antes — a regra dentro da Catraca

```

switch (regra) {           // String
  case "SALDO":
    pode = u.getSaldo()
           .compareTo(tarifa) ≥ 0;
    break;
  default:
    throw new IllegalStateException(..);
}

```

Depois — a regra é um objeto

```

interface RegraEntrada {
  Decisao avaliar(Usuario u, BigDecimal tarifa);
}
class SaldoSuficiente implements RegraEntrada { .. }

// na Catraca:
Decisao d = regra.avaliar(u, tarifa);
// na Montagem: "SALDO" → new SaldoSuficiente()

```

O texto "SALDO" / "TOLERA_NEGATIVO" continua chegando pela montagem — a assinatura de `Montagem.criarCatraca` não muda —, mas é traduzido para objeto **ali**, e a `Catraca` nunca mais o vê.

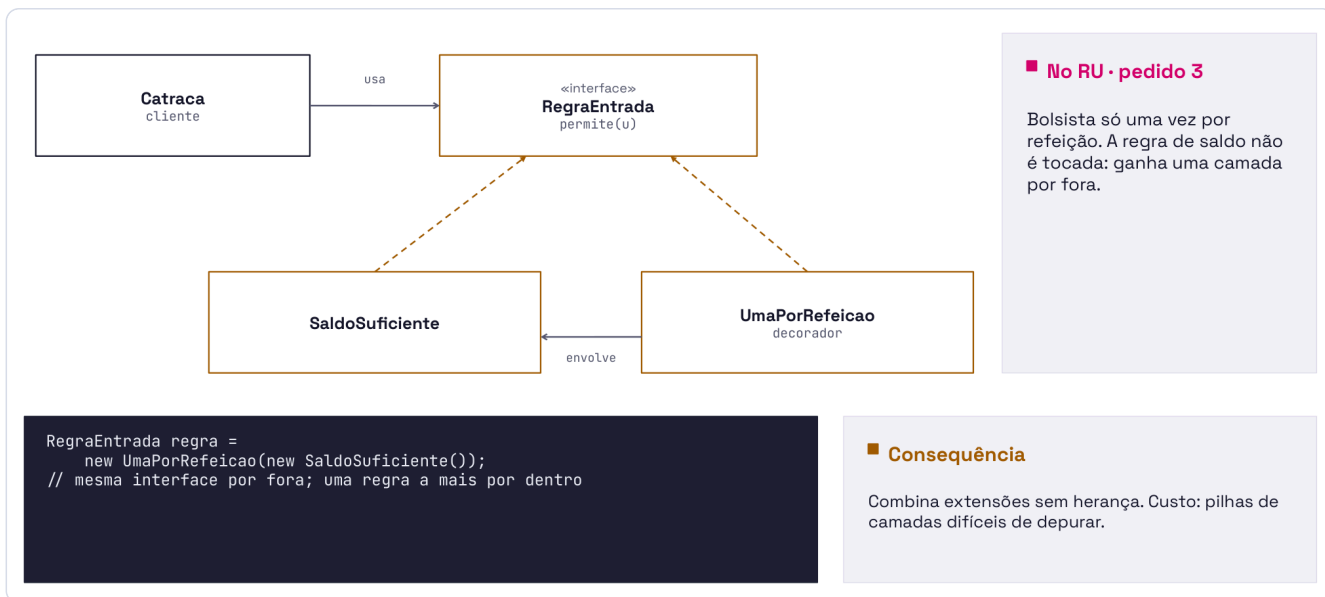
Consequências. **Ganha:** regra nova = classe nova + uma linha na montagem; cada regra testável isolada.

Paga: mais tipos; quem monta precisa saber qual estratégia escolher; um nível de indireção a mais para quem lê o fluxo.

E.5 · Pedido 3 · Decorador

Problema. O bolsista integral só pode passar uma vez por refeição — e a regra de saldo continua valendo para todos. Editar cada regra (`SaldoSuficiente`, `ToleraSaldoNegativo` ...) para acrescentar a restrição duplicaria código; criar subclasses para cada combinação (`SaldoSuficienteUmaVez` ...) explode o número de classes.

Decorador (GoF, estrutural). Intenção: acrescentar responsabilidades a um objeto dinamicamente, envolvendo-o com outro objeto que tem a **mesma interface**. Quem usa não percebe a diferença.



Decorator: a regra nova implementa RegraEntrada e envolve outra RegraEntrada.

```

class UmaPorRefeicao implements RegraEntrada {
    private final RegraEntrada interna;          // a regra envolvida – qualquer uma
    ...
    public Decisao avaliar(Usuario u, BigDecimal tarifa) {
        Decisao d = interna.avaliar(u, tarifa); // primeiro, a regra de sempre
        if (!d.liberar() || !u.isBolsista()) return d;
        // ... e só então a restrição nova
    }
}
// na Montagem: new UmaPorRefeicao(new SaldoSuficiente(), relógio)
  
```

O pedido 3 tem uma segunda metade: **provar que quem passa é o titular**. Esse é o segredo de outra peça — o **Identificador** — e pode ser resolvido do mesmo jeito: um identificador novo, com o mesmo contrato, que aceita o QR dinâmico assinado do app (veja `src/ru/app/QRcode.java`) e *envolve* o identificador por carteirinha para os pagantes. A **Catraca** não muda.

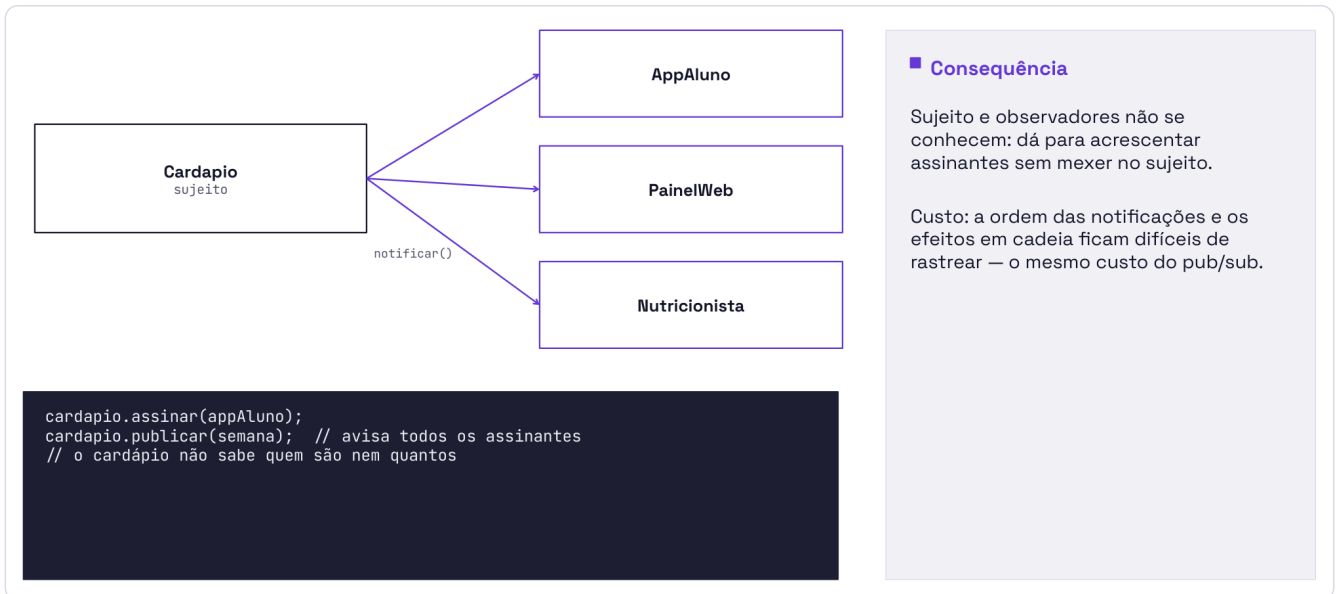
Decorator ou Proxy? Os dois envolvem um objeto com a mesma interface. A diferença está na **intenção**: o Decorador *acrescenta comportamento*; o Proxy *controla o acesso* ao objeto real (adiar a criação, cachear, checar permissão). Estrutura parecida não define o padrão; a intenção define.

Consequências. **Ganha:** combina extensões sem herança; a regra original fica intocada. **Paga:** pilhas de camadas difíceis de depurar; a ordem do empacotamento importa; aqui, a memória de “quem já passou” é local — duas catracas sem enlace não se enxergam (volte ao ADR-001).

E.6 · Pedido 4 · Observador

Problema. Para publicar a semana, o **Cardapio** cria e chama diretamente **AppAluno**, **PainelWeb** e **EmailGestao**. O DCE quer ser avisado também, e outros virão. Cada interessado novo reabre o **Cardapio**, que passa a conhecer todos os canais do sistema.

Observador (GoF, comportamental). Intenção: definir uma dependência um-para-muitos, de modo que, quando um objeto (o *sujeito*) muda de estado, todos os seus dependentes (os *observadores*) sejam avisados. O sujeito conhece só uma interface; a lista de interessados é montada de fora.



Observador: o Cardapio avisa quem assinou; não sabe quem são nem quantos.

Antes — o sujeito conhece os canais

```
class Cardapio {
    AppAluno app = new AppAluno();
    PainelWeb painel = new PainelWeb();
    EmailGestao email = new EmailGestao();

    void publicar(Semana s) {
        app.mostrarNotificacao(s);
        painel.atualizar(s);
        email.enviarResumo(s);
    }
}
```

Depois — o sujeito conhece uma interface

```
interface ObservadorCardapio {
    void semanaPublicada(Semana s);
}

class Cardapio {
    List<ObservadorCardapio> obs = new ArrayList<>();
    public void assinar(ObservadorCardapio o) { .. }
    public void publicar(Semana s) {
        for (var o : obs) o.semanaPublicada(s);
    }
}
```

Como a interface tem um único método abstrato, em Java ela pode ser satisfeita por uma **referência de método**: `cardapio.assinar(app::mostrarNotificacao)`. Assim os canais existentes não precisam nem mudar — a `MontagemCardapio` faz as inscrições. (As classes `java.util.Observer` e `Observable` existem, mas estão depreciadas desde o Java 9; não as usem.)

Consequências. **Ganha:** interessado novo = uma linha na montagem; o sujeito fica testável sem canais. **Paga:** o fluxo fica indireto (ler o `Cardapio` não diz quem é avisado); a ordem de aviso e o que fazer quando um observador falha precisam de uma política.

E.7 · Padrões que *não* entram — e por quê

PADRÃO	POR QUE NÃO AQUI
Singleton	tentador para a <code>Replica</code> ou a <code>Auditoria</code> , mas cria acesso global e dificulta os testes, que precisam de instâncias novas a cada caso. A montagem já garante “uma instância” sem o padrão.
Fachada	simplifica o acesso a um subsistema; nenhum pedido reclama de interface complicada.
Template Method	as regras de entrada não compartilham um esqueleto de passos; herança acoplaria mais que a composição do Strategy.
Factory Method / Abstract Factory	não há hierarquia de criadores nem família de objetos que precisem combinar; a fábrica simples resolve.

E.8 · Como os testes reconhecem um padrão sem saber o nome das classes

Os testes de aceitação não procuram `RegraEntrada` nem `PagaPixAdapter`. Eles verificam **efeitos estruturais** que qualquer boa solução produz:

SUÍTE	O QUE VERIFICA
<code>pedido1</code>	<code>ServicoCompra</code> , <code>Saldos</code> , <code>Auditoria</code> e <code>Recibo</code> não dependem de <code>ru.gateways</code> nem usam o vocabulário dos fornecedores; a compra funciona com <code>"cobrafacil"</code> e continua funcionando com <code>"pagapix"</code> .
<code>pedido2</code>	<code>TOLERA_NEGATIVO</code> funciona, inclusive no limite exato; o código da <code>Catraca</code> não contém <code>switch</code> , <code>"SALDO"</code> , <code>TOLERA</code> nem <code>compareTo</code> .
<code>pedido3</code>	bolsista entra com QR válido e sem pagar; carteirinha de bolsista, QR vencido, adulterado ou de outra chave não entram; uma vez por refeição; pagantes continuam entrando; tudo sem rede e em menos de 1 s; a <code>Catraca</code> não menciona bolsista, QR, HMAC nem refeição.
<code>pedido4</code>	<code>Cardapio</code> não depende de <code>ru.avisos</code> ; existe um método público que recebe um observador (uma interface com um único método que recebe a <code>Semana</code>); dois interessados novos são avisados.

Padrão é princípio aplicado com nome. Escolha pelo problema; o nome vem depois.

Especificação v2 do RU Digital

ID	REQUISITO	ISO 25010:2023
RF1	O usuário compra créditos pelo app, com pagamento via gateway externo.	Adequação funcional
RF2	A catraca autoriza a entrada por carteirinha ou QR code e desconta a tarifa do usuário.	Adequação funcional
RF3	A nutricionista publica o cardápio da semana; usuários o consultam no app e no site.	Adequação funcional
RF4	A tarifa depende do vínculo: aluno, servidor, visitante e bolsista integral (isento).	Adequação funcional
RF5	A gestão emite relatórios de consumo por refeição e por dia.	Adequação funcional
RNF1	No pico (11h–13h, até 800 usuários simultâneos), a compra conclui em ≤ 2 s no percentil 95.	Eficiência de desempenho → comportamento temporal
RNF2	Com o enlace indisponível, a catraca autoriza ou nega em ≤ 1 s a partir de dados locais e sincroniza as passagens em ≤ 5 min após o restabelecimento, sem perder nenhuma.	Confiabilidade → tolerância a falhas
RNF3	Toda alteração de saldo é autenticada e fica registrada em trilha de auditoria; o sistema não armazena dados de cartão.	Segurança → integridade, responsabilização
RNF4	Em falta de energia ou alarme de incêndio, a catraca libera a passagem mecanicamente.	Safety → falha segura
RNF5	Trocar o gateway de pagamento (novo contrato) altera um único arquivo da API de Créditos, sem tocar nas regras de crédito, e os testes existentes continuam passando. <i>(pedido 1)</i>	Manutenibilidade → modificabilidade
RNF6	Relatórios nunca expõem o consumo individual identificado; só agregados.	Segurança → confidencialidade
RNF8	A isenção do bolsista integral só pode ser usada pelo titular: carteirinha emprestada ou credencial copiada não autoriza a entrada, e cada bolsista passa no máximo uma vez por refeição. Vale também com o enlace caído, sem violar os tempos do RNF2. <i>(pedido 3)</i>	Segurança → autenticidade, não repúdio
RNF9	A regra de entrada é escolhida na montagem da catraca, a partir da configuração: <code>SALDO</code> (entra quem tem saldo $\geq$ tarifa) ou <code>TOLERA_NEGATIVO</code> (entra quem fica com saldo $\geq$ -R\$ 5,00 após a tarifa). Incluir uma regra nova não altera a classe <code>Catraca</code> . <i>(pedido 2)</i>	Manutenibilidade → modificabilidade
RNF10	Quando a semana é publicada, app, painel e e-mail da gestão são avisados. Um novo interessado pode ser acrescentado sem alterar a classe <code>Cardapio</code> , que não depende de nenhum canal de aviso. <i>(pedido 4)</i>	Manutenibilidade → modificabilidade

 exemplo resolvido  atividade. A numeração pula o RNF7 porque um requisito levantado na revisão (troca do fabricante da catraca) foi retirado do escopo desta versão.

Referências

- BASS, L.; CLEMENTS, P.; KAZMAN, R. *Software Architecture in Practice*. 4. ed. Addison-Wesley, 2021 — cenários de qualidade.
- KAZMAN, R.; KLEIN, M.; CLEMENTS, P. *ATAM: Method for Architecture Evaluation*. CMU/SEI-2000-TR-004, 2000 — pontos de sensibilidade e de trade-off.
- ISO/IEC 25010:2023 — [iso.org/standard/78176.html](https://www.iso.org/standard/78176.html); resumo aberto em quality.arc42.org.
- NYGARD, M. [Documenting Architecture Decisions](#), 2011.
- BROWN, S. [The C4 model for visualising software architecture](#).
- PARNAS, D. L. On the Criteria to Be Used in Decomposing Systems into Modules. *CACM*, 15(12), 1972. [doi:10.1145/361598.361623](https://doi.org/10.1145/361598.361623).
- FORD, N.; PARSONS, R.; KUA, P. *Building Evolutionary Architectures*. O'Reilly, 2017 — funções de aptidão.
- GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994 (ed. brasileira: *Padrões de Projeto*, Bookman, 2000).
- VALENTE, M. T. *Engenharia de Software Moderna*, caps. 5 (princípios), 6 (padrões) e 7 (arquitetura) — engsoftmoderna.info.
- Refactoring.Guru — catálogo de padrões com exemplos em Java: refactoring.guru/pt-br/design-patterns.
- BRASIL. Lei nº 13.709/2018 (LGPD), art. 5º, II, e art. 11 — dado biométrico como dado pessoal sensível. planalto.gov.br.